

# SLIOT — Benthic Guardian

## Backend Technical Documentation

Team Terronix · Smart Lankan IoT 2026

August 2026

Backend Version 1.0 · FastAPI · PostgreSQL · TensorFlow · Docker · Render

## 1. Project Overview

SLIOT (Smart Lankan IoT) is a real-time coral reef monitoring platform built by Team Terronix. It collects water temperature data from ESP32 sensors placed at reef sites along Sri Lanka's coastline. The platform predicts future temperature changes and coral bleaching risk using two machine learning models, then shows the results in a live web dashboard.

This document explains how the backend was designed, the decisions we made, and the technical details of each part. We use simple language so anyone on the team can understand it.

### 1.1 What the System Does

- Receives live temperature readings from underwater ESP32 sensors via HTTP POST.
- Stores all readings and forecasts in a managed PostgreSQL database (Supabase).
- Runs a Physics-Informed Neural Network (PINN) to forecast the next 7 days of temperature, every 6 hours.
- Runs an ANN-LSTM model to forecast SST and bleaching risk at +1, +3, and +7 days ahead.
- Sends real-time bleaching alerts to the dashboard via WebSocket when temperature exceeds 31°C.
- Controls access using JWT tokens and role-based permissions (admin vs regular user).

### 1.2 System Context

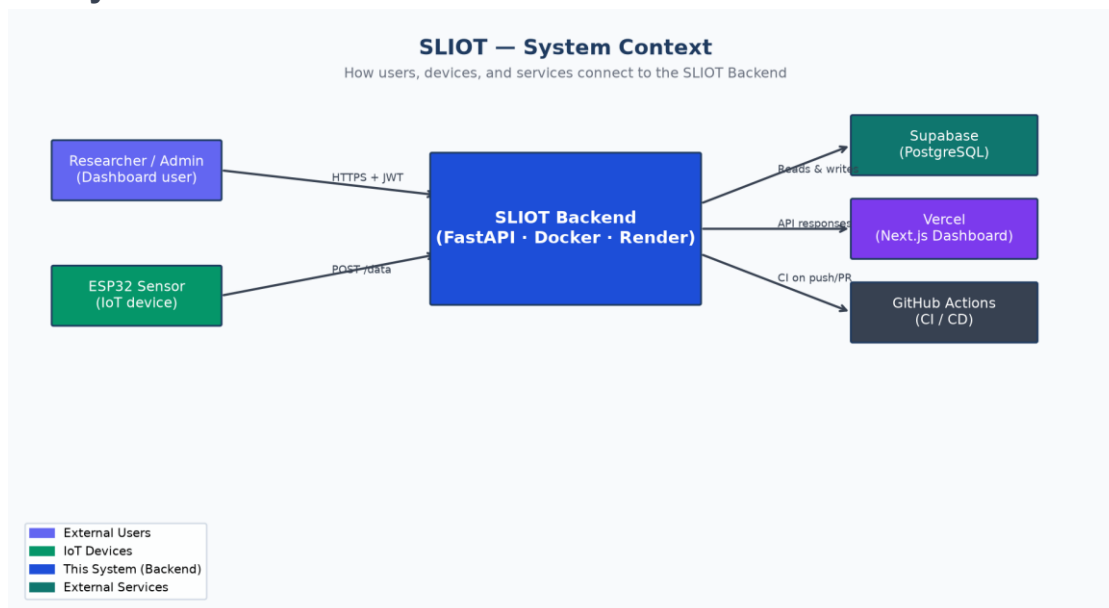


Figure 1 — System Context: how actors and services connect to the backend

## 2. System Architecture

The backend is a single FastAPI application running inside a Docker container on Render. It has several internal components that work together. The diagram below shows how they fit.

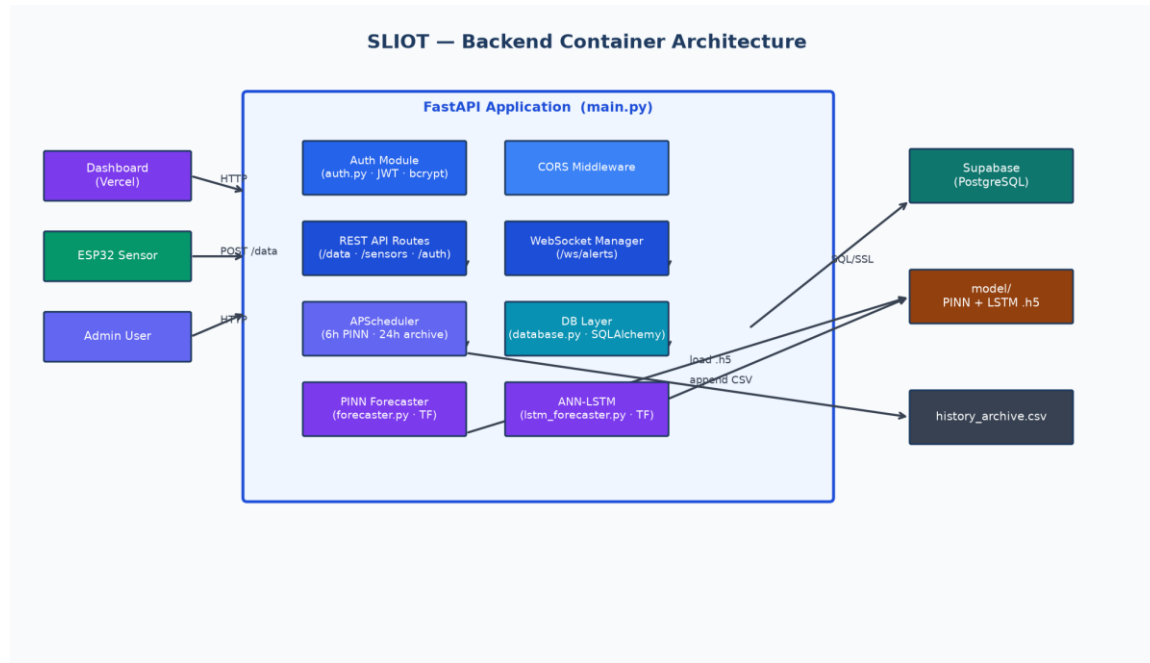


Figure 2 — Container architecture: internal components and external dependencies

### 2.1 Key Components

| Component       | File(s)                  | What it does                                                       |
|-----------------|--------------------------|--------------------------------------------------------------------|
| FastAPI App     | main.py                  | Registers all routes, lifespan startup, CORS, WebSocket manager    |
| Auth Module     | auth.py                  | JWT creation/validation, bcrypt password hashing, role checks      |
| Database Layer  | database.py              | SQLAlchemy engine, session factory, URL normalization for Supabase |
| ORM Models      | models.py                | Defines all 6 database tables as Python classes                    |
| Scheduler       | scheduler.py             | APScheduler: runs PINN every 6h and data archival every 24h        |
| PINN Forecaster | model/forecaster.py      | Loads pinn_model_best.h5, generates 168-hour hourly predictions    |
| ANN-LSTM        | model/lstm_forecaster.py | Loads ann_lstm_L60_best.h5, generates +1/+3/+7 day forecasts       |
| Model Paths     | model_paths.py           | Resolves model/ directory for both monorepo and standalone deploy  |
| Sync Script     | sync_model_assets.py     | Copies model weights + datasets into backend/model/ for Docker     |

## 2.2 File Structure

The main files in the backend repository are:

- `main.py` — All API routes, app factory, lifespan
- `auth.py` — JWT and bcrypt utilities
- `database.py` — Engine, session, URL handling
- `models.py` — SQLAlchemy ORM (6 tables)
- `scheduler.py` — Background jobs (PINN + archive)
- `model_paths.py` — Resolves model directory
- `model/` — ML weights, datasets, forecasters
- `tests/` — 29 pytest tests
- `.github/` — GitHub Actions CI workflow
- `Dockerfile` — Docker image (python:3.11-slim-bookworm)
- `render.yaml` — Render blueprint configuration

### 3. Database Design

The database runs on Supabase (managed PostgreSQL). There are 6 tables. SQLAlchemy is used as the ORM so the code does not write raw SQL.

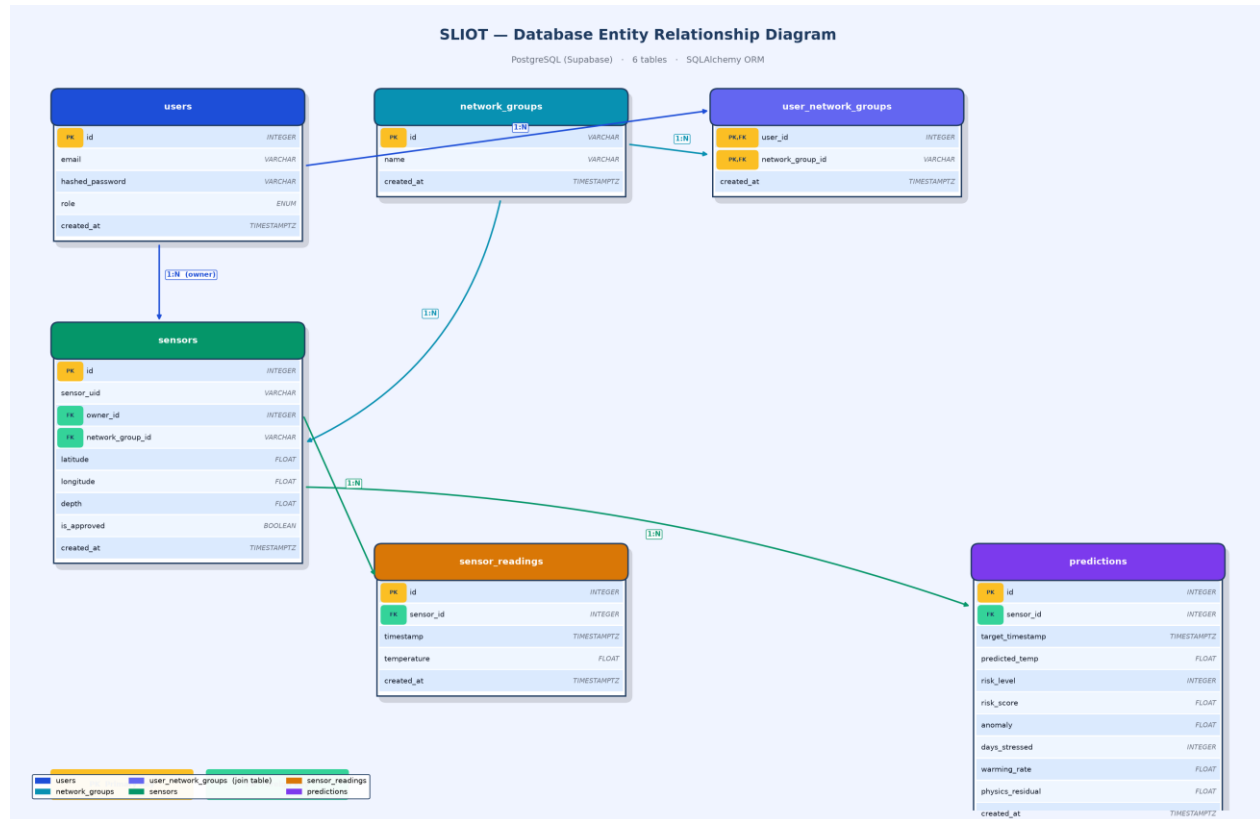


Figure 3 — Entity Relationship Diagram (ERD): all tables and their relationships

#### 3.1 Table Descriptions

| Table               | Description                                                                                                                                                              |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| users               | Stores every account. Role is either 'admin' or 'user'. Password is stored as a bcrypt hash, never in plain text.                                                        |
| network_groups      | A logical group of sensors (e.g. 'Hikkaduwa Reef Network'). ID is a string like 'ng_abc123'. Users belong to one or more groups.                                         |
| user_network_groups | Join table connecting users and network groups. A user can be in many groups; a group can have many users (many-to-many).                                                |
| sensors             | Each physical ESP32 device. Stores GPS coordinates, depth, the owner user, and a flag for approval. A sensor must be approved by an admin before it can POST data.       |
| sensor_readings     | One row per temperature reading received. No deduplication — full-resolution timestamps. Rows older than 30 days are archived to CSV and deleted.                        |
| predictions         | PINN-generated 7-day (168 rows) forecast per sensor. Refreshed every 6 hours. Includes temperature, risk level (0/1/2), risk score (0–1), anomaly, and physics residual. |

## 3.2 Design Decisions

### Why no hourly deduplication?

An earlier version had a unique constraint so only one reading per sensor per hour was stored. This caused 409 Conflict errors when sensors posted more frequently. We removed it (via migration) to store every reading at full resolution, which gives richer data for the PINN.

### Why Supabase session pooler (port 5432)?

Supabase offers two pooler types. The transaction pooler (port 6543) does not support prepared statements, which SQLAlchemy uses by default. The session pooler (port 5432) behaves like a normal PostgreSQL connection and works without extra configuration. We also URL-encode special characters in the password to avoid connection string parsing errors.

### Why soft pool on Render?

Render Starter has limited RAM. We set pool\_size=5 and max\_overflow=5 (10 total connections) to avoid overwhelming Supabase's free tier connection limit and to keep memory usage low.

## 4. API Design

The API is built with FastAPI. All endpoints return JSON. Most endpoints require a JWT bearer token. A few (health, register, login) are public.

### 4.1 Endpoint Map

| Method | Path                        | Auth        | Purpose                                                |
|--------|-----------------------------|-------------|--------------------------------------------------------|
| GET    | /                           | None        | Health check — returns status, version, database state |
| POST   | /auth/register              | None        | Create user account (role = 'user')                    |
| POST   | /auth/token                 | None        | Login — returns JWT bearer token                       |
| GET    | /auth/me                    | User        | Return current user profile                            |
| GET    | /sensors                    | User        | List sensors (admin: all, user: own network)           |
| POST   | /admin/register-sensor      | Admin       | Register and approve a new sensor                      |
| POST   | /admin/users                | Admin       | Create a user account (admin-only)                     |
| POST   | /admin/network-groups       | Admin       | Create a new network group                             |
| GET    | /admin/users                | Admin       | List all user accounts                                 |
| POST   | /data                       | None        | Ingest sensor reading (sensor_uid + temperature)       |
| GET    | /sensors/{id}/readings      | User        | Recent readings history for a sensor                   |
| GET    | /sensors/{id}/forecast      | User        | PINN forecast (168h) for a sensor                      |
| GET    | /sensors/{id}/lstm-forecast | User        | ANN-LSTM forecast (+1/+3/+7d) for a sensor             |
| GET    | /api/lstm-forecast          | User        | ANN-LSTM forecast by reef site location                |
| GET    | /api/stats                  | User        | Dashboard KPIs (24h avg/max temp, counts)              |
| GET    | /api/risk-summary           | User        | Risk counts (healthy/warning/danger) across network    |
| GET    | /api/sst                    | User        | Sea Surface Temperature historical data                |
| GET    | /api/dhw                    | User        | Degree Heating Week historical data                    |
| GET    | /api/latest-readings        | User        | Most recent reading per sensor                         |
| WS     | /ws/alerts                  | JWT (query) | Push bleaching alerts when temp > 31°C                 |

## 4.2 Request Sequences

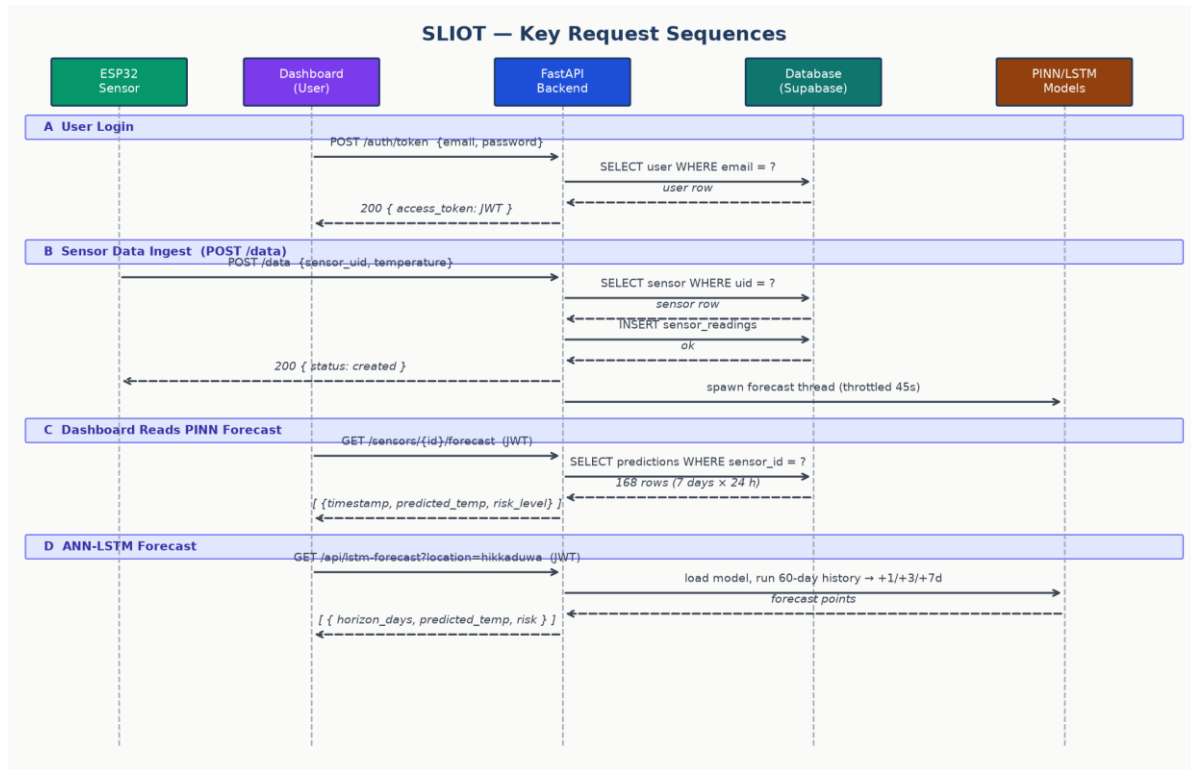


Figure 4 — Key request sequences: login, sensor ingest, PINN forecast, LSTM forecast

## 5. Authentication & Authorization

The backend uses JSON Web Tokens (JWT) for stateless authentication. Every protected endpoint uses a FastAPI dependency that decodes and validates the token.

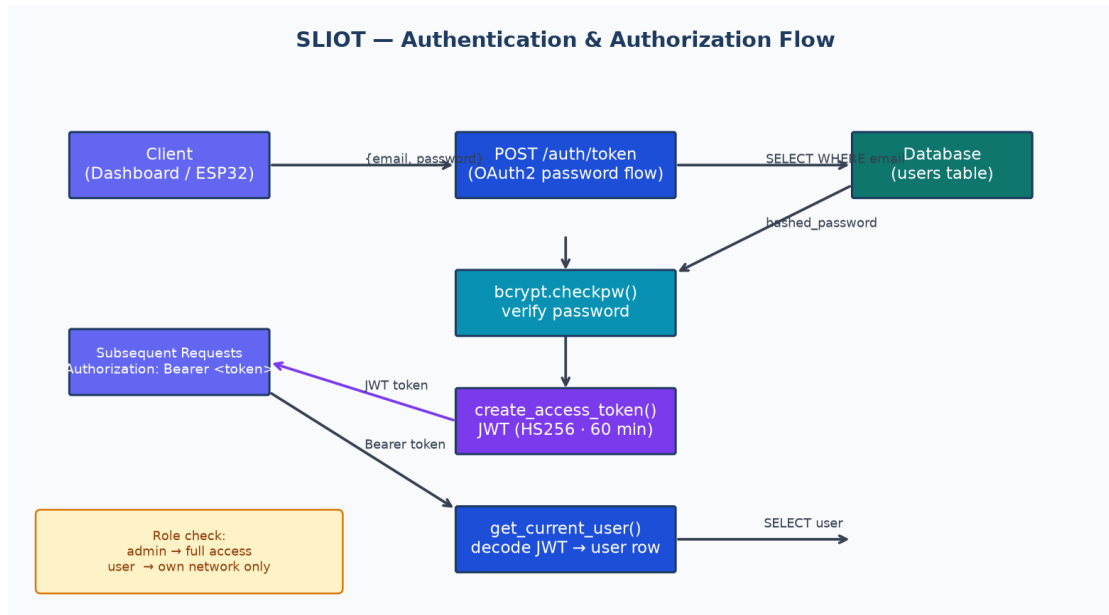


Figure 5 — Authentication flow: login, bcrypt verify, JWT issue, role check

### 5.1 How It Works

- User sends email + password to POST /auth/token.
- Backend looks up the user, then calls `bcrypt.checkpw()` to verify the password hash.
- On success, it creates a JWT signed with `SECRET_KEY` (HS256). Token lifetime is 60 minutes.
- Client stores the token and sends it as 'Authorization: Bearer <token>' on every request.
- The `get_current_user()` dependency decodes the token and loads the user from the database.
- The `get_admin_user()` dependency additionally checks that the role is 'admin'.

### 5.2 Why bcrypt directly (not passlib)?

We started with `passlib` for password hashing. However, `passlib` is incompatible with `bcrypt` version 4.x and above on Linux/Docker. When deployed to Render, every login and register returned a 500 Internal Server Error. We fixed this by calling `bcrypt` directly (`bcrypt.hashpw` and `bcrypt.checkpw`) which works correctly with any `bcrypt` version.

### 5.3 Role-Based Access Control

| Role  | Who                  | What they can access                                 |
|-------|----------------------|------------------------------------------------------|
| admin | Team / project owner | All endpoints, all sensors, all users, all networks  |
| user  | Reef site owner      | Only sensors and readings in their own network group |

## 6. Machine Learning Models

The backend integrates two machine learning models. They serve different purposes and complement each other. Both use TensorFlow and are loaded lazily to avoid slowing down the API start-up.

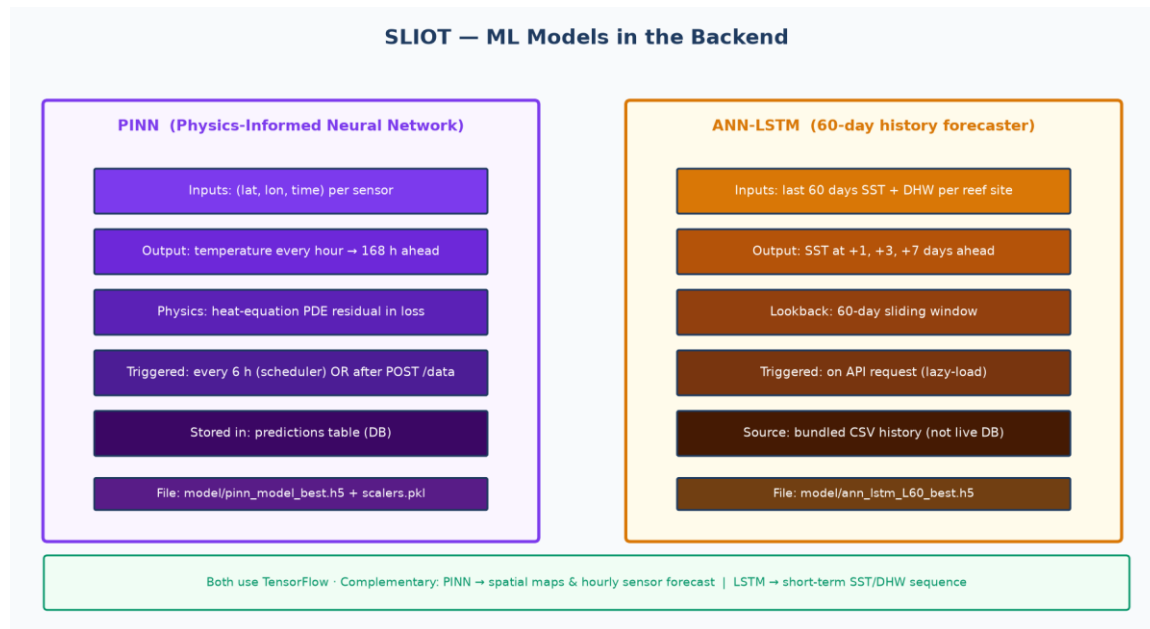


Figure 6 — ML models overview: PINN (spatial/hourly) vs ANN-LSTM (short-term sequence)

### 6.1 PINN — Physics-Informed Neural Network

The PINN was trained to predict ocean temperature at any (latitude, longitude, time) point around the reef sites. It uses the heat-equation PDE as a physics constraint in its training loss, which makes it physically realistic and better at extrapolating into the future.

- Input: (latitude, longitude, time\_hours)
- Output: temperature prediction (°C) for that point and time
- Forecast horizon: 168 hours (7 days) ahead, one row per hour
- Trigger: every 6 hours by the scheduler, or immediately after a POST /data (throttled to once every 45 seconds)
- Storage: predictions table in the database (old rows replaced on each run)
- Files: model/pinn\_model\_best.h5, model/scalars.pkl, model/sensor\_info.pkl

## 6.2 ANN-LSTM — Short-Term Sequence Forecaster

The ANN-LSTM uses a 60-day sliding window of Sea Surface Temperature (SST) and Degree Heating Weeks (DHW) to predict the next 1, 3, and 7 days. This model focuses on recent patterns rather than spatial physics, so it catches short-term warming trends that the PINN might smooth over.

- Input: last 60 days of SST and DHW for a reef site
- Output: predicted SST, DHW, anomaly, and bleaching risk at +1, +3, +7 days
- Trigger: on API request (lazy-loaded — not stored in the DB)
- Source data: bundled CSV history files (not live sensor\_readings)
- Locations: hikkaduwa, kalpitiya, passikudha, south\_east, trinco
- Files: model/ann\_lstm\_L60\_best.h5, model/sliot\_dataset/<location>/sst\_full.csv

## 6.3 Why Two Models?

|                    | PINN                              | ANN-LSTM                     |
|--------------------|-----------------------------------|------------------------------|
| What it predicts   | Temperature at any reef location  | SST + DHW for 5 named sites  |
| Time range         | 7 days (168 h hourly)             | +1, +3, +7 days              |
| Data source        | Live sensor readings              | 60-day CSV history           |
| Physics constraint | Yes (heat equation PDE)           | No (data-driven only)        |
| Good at            | Spatial maps, per-sensor forecast | Short-term SST trend changes |
| Stored in DB?      | Yes (predictions table)           | No (computed on demand)      |

## 7. Background Scheduler

The backend runs two background jobs using APScheduler. They start when the API starts and run continuously while the server is up.

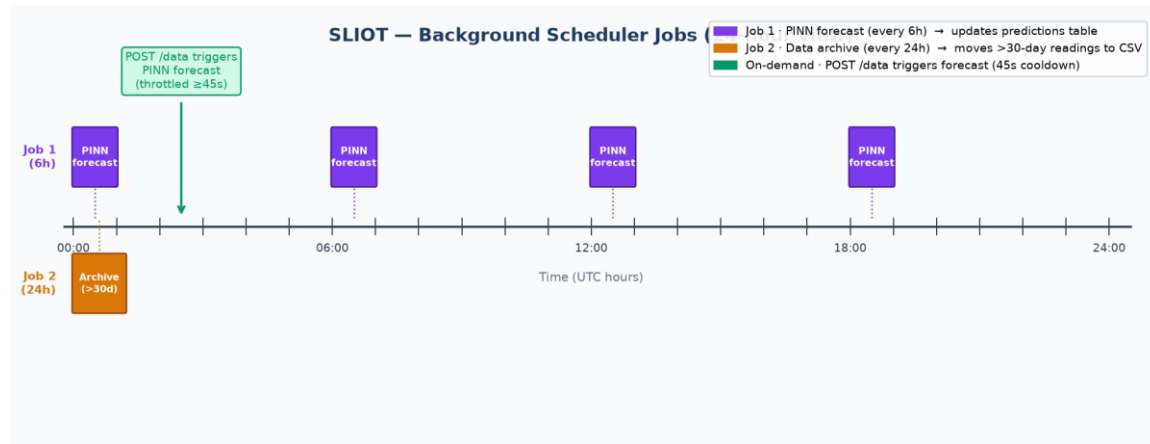


Figure 7 — Scheduler timeline: Job 1 (PINN every 6h), Job 2 (archive every 24h), and on-demand trigger

### 7.1 Job 1 — PINN Forecast (every 6 hours)

- Fetches the last 48 hours of readings for every approved sensor.
- Runs the PINN model for each sensor to generate a 168-hour forecast.
- Deletes old predictions for that sensor and writes fresh ones to the database.
- If a sensor has no readings in the last 48 hours, it is skipped.
- Also triggered by POST /data, but no more than once every 45 seconds.

### 7.2 Job 2 — Data Archival (every 24 hours)

- Finds sensor\_readings rows older than 30 days.
- Appends them to history\_archive.csv (on disk).
- Deletes them from the database to keep the 'hot' table fast.
- If the CSV does not exist, it creates it with a header.

### 7.3 Disabling the Scheduler

Set the environment variable `DISABLE_SCHEDULER=1` to skip APScheduler on startup. This is useful when running tests (done automatically via `conftest.py`) or when the Render instance is too small to handle TensorFlow in-process. The API continues to serve all non-forecast endpoints normally.

## 8. Real-Time Bleaching Alerts (WebSocket)

When a sensor posts a temperature at or above 31°C (the coral bleaching threshold), the backend immediately broadcasts a `bleaching_alert` message to all connected WebSocket clients.

- Clients connect to `/ws/alerts?token=<jwt>` using the `wss://` protocol.
- Admin users receive alerts from every sensor.
- Regular users receive alerts only from sensors in their own network group.
- If the client sends a plain HTTP GET to `/ws/alerts` (no Upgrade header), a 200 response with a helpful message is returned.
- All connected clients also receive a `reading_new` event on every new POST `/data`.

## 9. Deployment

The backend is deployed as a Docker container on Render (Singapore region). The database runs on Supabase (managed PostgreSQL). The dashboard is on Vercel.

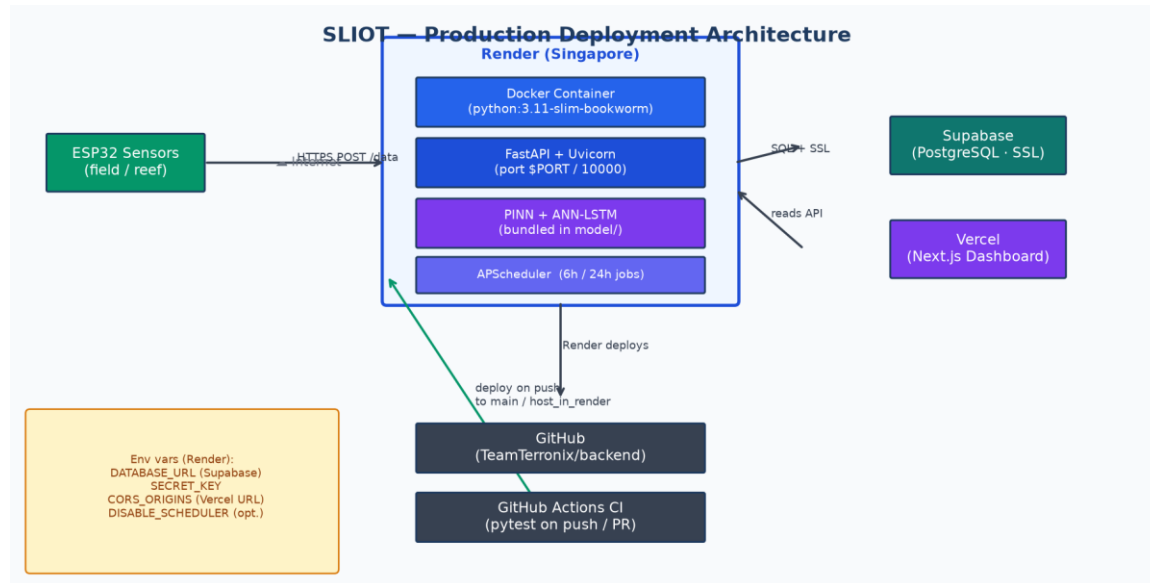


Figure 8 — Production deployment architecture: Render, Supabase, Vercel, GitHub

### 9.1 How Deployment Works

- Code is pushed to GitHub (TeamTerrorix/backend-Benthic-Guardian).
- Render picks up the push on the configured branch (main or host\_in\_render).
- It builds the Docker image using Dockerfile (python:3.11-slim-bookworm base).
- On startup, docker-entrypoint.sh checks the DB connection, creates tables if needed, then starts uvicorn on \$PORT (default 10000).
- Render scans for an open port. If it finds one within ~90 seconds, the deploy succeeds.

### 9.2 Environment Variables

| Variable                    | Required? | Description                                                                 |
|-----------------------------|-----------|-----------------------------------------------------------------------------|
| DATABASE_URL                | Yes       | Supabase session pooler URI (port 5432). URL-encode password special chars. |
| SECRET_KEY                  | Yes       | Long random string for signing JWT tokens. Never share or commit.           |
| ALGORITHM                   | No        | JWT algorithm. Defaults to HS256.                                           |
| ACCESS_TOKEN_EXPIRE_MINUTES | No        | Token lifetime. Defaults to 60 minutes.                                     |
| CORS_ORIGINS                | Yes       | Comma-separated list of allowed frontend URLs (e.g. your Vercel URL).       |
| CORS_ORIGIN_REGEX           | No        | Regex for CORS origins (e.g. https://.*\.vercel\.app).                      |
| CUDA_VISIBLE_DEVICES        | No        | Set to -1 to force TensorFlow to use CPU (important on Render).             |
| DISABLE_SCHEDULER           | No        | Set to 1 to skip APScheduler. Useful                                        |

|      |    |                                                   |
|------|----|---------------------------------------------------|
|      |    | for small instances or debugging.                 |
| PORT | No | Port for uvicorn. Render sets this automatically. |

## 9.3 Known Deployment Issues

### Port scan timeout

If the API process does not open a port within ~90 seconds, Render reports 'No open ports detected'. This usually means TensorFlow or the PINN is loading at startup and taking too long. Fix: set `DISABLE_SCHEDULER=1` so the model does not load at boot.

### OOM crash (out of memory)

TensorFlow + PINN and ANN-LSTM together use significant RAM. On Render Starter or Free tier, the process can be killed by the OS. This appears as a random crash with no Python traceback, just 'Killed' in the logs. Fix: upgrade instance size, or run forecasts in a separate worker service.

### Supabase paused

The free Supabase project pauses after 7 days of inactivity. The API will fail all DB operations until you resume it from the Supabase dashboard. Redeploy or restart the Render service after resuming.

## 10. Testing & Continuous Integration

The backend has 29 automated tests written with pytest. All tests run in CI on every push and pull request via GitHub Actions.

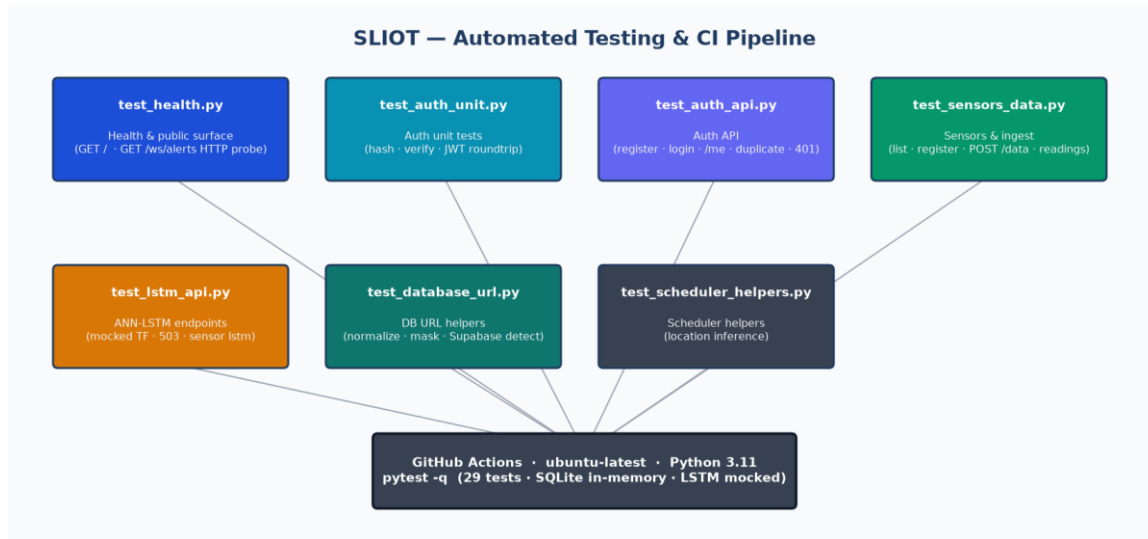


Figure 9 — Test structure and GitHub Actions CI pipeline

### 10.1 Test Design Decisions

#### SQLite in-memory for speed

Tests use a shared in-memory SQLite database instead of the real Supabase. This makes tests fast (under 10 seconds) and does not require any network access. The `DATABASE_URL` is overridden to `'sqlite://'` before the app imports.

#### Mocked LSTM / PINN

TensorFlow is not installed in the test environment (`requirements-dev.txt` has no TF). The `_get_lstm()` function in `main.py` is monkeypatched with a `MagicMock` that returns a fixed list of forecast points. This lets us test the API response shape without loading any model weights.

#### DISABLE\_SCHEDULER=1

`conftest.py` sets this env var so `APScheduler` does not start, and no PINN jobs run during tests. The `_schedule_forecast_job_after_reading()` function is also patched to a no-op on the client fixture.

### 10.2 Test Summary

| Test file         | Count | What is tested                                                        |
|-------------------|-------|-----------------------------------------------------------------------|
| test_health.py    | 2     | GET / returns ok + database ok.<br>/ws/alerts HTTP probe returns 200. |
| test_auth_unit.py | 3     | hash_password, verify_password,<br>JWT encode/decode roundtrip.       |
| test_auth_api.py  | 5     | Register, login, /me, duplicate email<br>400, wrong password 401.     |

|                           |   |                                                                           |
|---------------------------|---|---------------------------------------------------------------------------|
| test_sensors_data.py      | 8 | List sensors, admin register, unapproved block, ingest, readings history. |
| test_lstm_api.py          | 4 | LSTM requires auth, mock forecast, 503 on missing weights, per-sensor.    |
| test_database_url.py      | 4 | URL normalize, quote-strip, Supabase detect, mask (password hidden).      |
| test_scheduler_helpers.py | 3 | Location inference from UID prefix and from GPS coordinates.              |

## 10.3 Running Tests Locally

**Install dev dependencies and run pytest:**

```
pip install -r requirements-dev.txt  
pytest -q
```

## 11. Key Design Decisions

### 1. FastAPI over Flask or Django

FastAPI gives automatic OpenAPI docs, Pydantic validation, async support, and is faster. Django was too heavy for a pure API service. Flask lacks built-in validation.

### 2. bcrypt directly (not passlib)

passlib's bcrypt wrapper is incompatible with `bcrypt>=4.0.0` on Linux/Docker (used on Render). Calling `bcrypt.hashpw()` and `bcrypt.checkpw()` directly works with any version.

### 3. JWT (stateless) over session cookies

The dashboard and ESP32 sensors both need auth. Stateless JWT works across domains without a shared session store. Tokens expire after 60 minutes.

### 4. Supabase (managed PostgreSQL) over self-hosted

Free managed database with automatic backups, SSL, and a pooler. Avoids running and paying for a separate database server. Matches Render's free/starter tier budget.

### 5. Session pooler port 5432 over transaction pooler 6543

The transaction pooler disables prepared statements, which SQLAlchemy uses by default. Session pooler behaves like a normal PostgreSQL connection and needs no extra config.

### 6. Docker on Render over buildpack

Docker gives full control over the runtime environment, including TensorFlow dependencies and model files. Buildpacks cannot reliably install TF or bundle large binary files.

### 7. PINN + ANN-LSTM (two models)

PINN handles spatial mapping and per-sensor hourly forecasts using physics constraints. ANN-LSTM handles short-term SST/DHW trends using sequence data. They cover different time scales and data types so both are useful.

### 8. Lazy model loading

Both models are loaded on first use, not at import time. This keeps API startup fast and avoids loading TF when it is not needed (e.g. during tests).

## 9. Removing hourly deduplication

The original unique constraint on (sensor\_id, hour) caused 409 errors when sensors posted more than once per hour. Removing it stores full-resolution data, which is more accurate and flexible for ML training.

## 10. In-memory SQLite for tests

Tests should not require a real database connection. SQLite in-memory is fast, isolated, and works on any machine including GitHub Actions CI runners with no DB server.

## 12. Known Limitations & Future Work

### 12.1 Current Limitations

- TensorFlow uses a lot of RAM. Running both PINN and ANN-LSTM on a Render Starter instance can cause OOM crashes.
- The scheduler runs inside the web process. Heavy forecast jobs can delay API responses.
- ANN-LSTM uses bundled CSV history, not live sensor\_readings. It will not reflect brand-new sensor sites without retraining.
- WebSocket connections are stored in memory. If the server restarts, all connected clients lose their connection.
- No rate limiting on POST /data. A malfunctioning sensor could flood the database.

### 12.2 Recommended Future Improvements

- Move forecast jobs to a separate Render background worker to isolate TF memory from the web API.
- Add rate limiting (e.g. slowapi) on sensor data ingestion.
- Persist WebSocket sessions via Redis for multi-instance deployments.
- Retrain ANN-LSTM periodically on new sensor data to keep forecasts accurate.
- Add pagination to /sensors/{id}/readings for large datasets.
- Add structured logging with request IDs for easier debugging on Render.

### 13. Related Repositories & Links

| Item               | Link / Notes                                                                                                            |
|--------------------|-------------------------------------------------------------------------------------------------------------------------|
| Backend repo       | <a href="https://github.com/TeamTerronix/backend-Benthic-Guardian">github.com/TeamTerronix/backend-Benthic-Guardian</a> |
| Dashboard repo     | Next.js frontend (separate repo)                                                                                        |
| Model repo         | PINN + ANN-LSTM training notebooks (separate repo)                                                                      |
| Live API           | <a href="https://backend-benthic-guardian.onrender.com">https://backend-benthic-guardian.onrender.com</a>               |
| API docs (Swagger) | <a href="https://backend-benthic-guardian.onrender.com/docs">https://backend-benthic-guardian.onrender.com/docs</a>     |
| Database           | Supabase (Singapore) — session pooler port 5432                                                                         |
| Deployment guide   | RENDER_DEPLOYMENT.md in the backend repo                                                                                |